

7. Content Coupling

- ▶ Content Coupling exists among two modules if they share code, e.g., a branch from one module into another module.

Content Coupling

Module A – Manages customer records

Module B – Needs to update a customer's balance

Instead of calling a function in Module A, Module B directly changes variables inside Module A.

Module A

```
balance = 5000
```

```
def show_balance():
```

```
    print("Current balance:", balance)
```

Module B

```
import moduleA
```

```
moduleA.balance -= 1000 # Directly modifying internal data of Module A
```

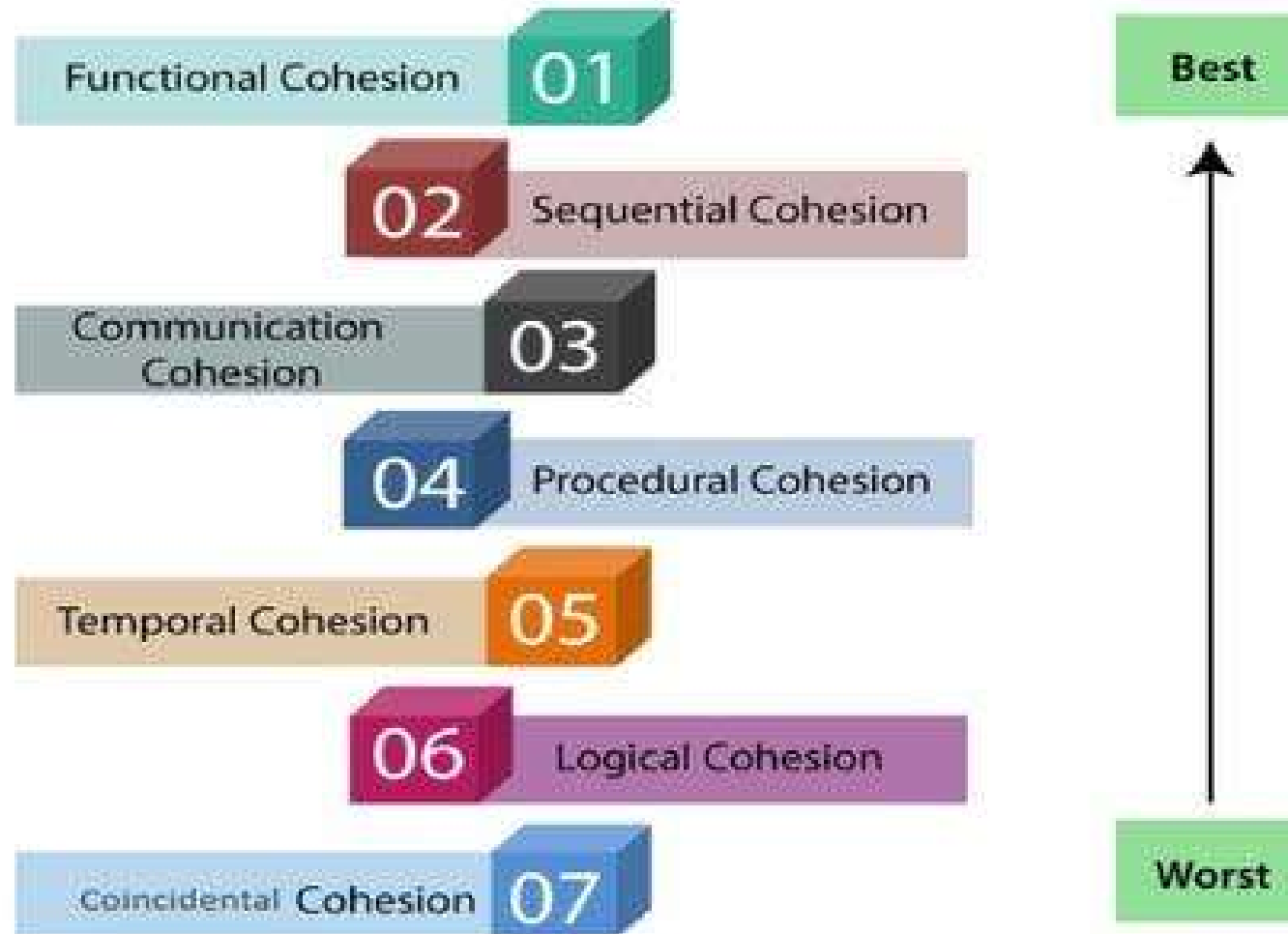
```
moduleA.show_balance()
```

Cohesion

- ▶ Cohesion defines to the degree to which the elements of a module belong together. Thus, cohesion measures the strength of relationships between pieces of functionality within a given module. For example, in highly cohesive systems, functionality is strongly related. A good software design will have high cohesion.
- 

Types of Modules Cohesion

Types of Modules Cohesion



1. Functional Cohesion

- ▶ Functional Cohesion is said to exist if the different elements of a module, cooperate to achieve a single function.

```
def withdraw_cash(account, amount):  
    # Check balance  
    if account['balance'] < amount:  
        return "Insufficient balance"  
  
    # Deduct amount  
    account['balance'] -= amount  
  
    # Record transaction  
    account['transactions'].append({'type': 'withdrawal', 'amount': amount})  
  
    # Print confirmation  
    return f"{amount} withdrawn successfully"
```

2. Sequential Cohesion

- ▶ A module is said to possess sequential cohesion if the elements of a module form the components of the sequence, where the output from one component of the sequence is input to the next.

```
def process_order(order):  
    valid_order = validate_order(order)  
    total_price = calculate_total(valid_order)  
    invoice = create_invoice(total_price)  
    return invoice
```

```
function processDataPipeline(filePath) {  
    const rawData = readFile(filePath); // string  
    const parsedData = parseJSON(rawData); // object  
    saveToDatabase(parsedData); // uses object  
}
```

A data processing function:
`readDataFromFile()` returns a string, which is passed to
`parseJSON()` which returns an object, which is passed to
`saveToDatabase()`

3. Communicational Cohesion

- ▶ A module is said to have communicational cohesion, if all tasks of the module refer to or update the same data structure, e.g., the set of functions defined on an array or a stack.

```
class Customer {  
  constructor(data) { this.data = data; }  
  
  updateAddress(newAddress) {  
    this.data.address = newAddress;  
  }  
  
  addPurchase(purchase) {  
    this.data.purchases.push(purchase);  
  }  
  getPurchaseHistory() {  
    return this.data.purchases;  
  }  
}
```

A `Customer` class with methods like `updateAddress(newAddress)`, `getPurchaseHistory()`, and `applyDiscount()`. All methods manipulate the data of a single `Customer` object.

4. Procedural Cohesion

- ▶ A module is said to be procedural cohesion if the set of purpose of the module are all parts of a procedure in which particular sequence of steps has to be carried out for achieving a goal, e.g., the algorithm for decoding a message.

```
def giki_student():
```

```
    #1: Wake up
```

```
    wake_up()
```

```
    #2: Brush teeth
```

```
    brush_teeth()
```

```
    # 3: Take shower
```

```
    take_shower()
```

```
    # 4: Eat breakfast
```

```
    eat_breakfast()
```

```
    # 5: Attending class
```

```
    attend_class()
```

```
def handle_request():
```

```
    authenticate_user() # Step 1
```

```
    log_access()        # Step 2
```

```
    open_resource()     # Step 3
```

5. Temporal Cohesion

- ▶ When a module includes functions that are associated by the fact that all the methods must be executed in the same time, the module is said to exhibit temporal cohesion.

Example

```
function initialize() {  
  loadConfigFile();  
  connectToDatabase();  
  setupErrorHandlers();  
  initializeCache();  
}
```

6. Logical Cohesion

- ▶ A module is said to be logically cohesive if all the elements of the module perform a similar operation. For example Error handling, data input and data output, etc.

```
class IOModule {  
  readFromKeyboard() { ... }  
  readFromFile(path) { ... }  
  writeToScreen(text) { ... }  
  writeToPrinter(data) { ... }  
}
```

```
class MathUtility:  
  def add(a, b):  
    return a + b  
  
  def subtract(a, b):  
    return a - b  
  
  def multiply(a, b):  
    return a * b  
  
  def divide(a, b):  
    if b != 0:  
      return a / b  
    return None
```


7. Coincidental Cohesion

- ▶ A module is said to have coincidental cohesion if it performs a set of tasks that are associated with each other very loosely, if at all.

Example

```
def do_everything():  
    print("Hello World")      # Task 1  
    calculate_tax(1000)      # Task 2  
    play_notification_sound() # Task 3  
    send_email("example@test.com") # Task 4
```

Differentiate between Coupling and Cohesion

S.No	Coupling	Cohesion
1	Cohesion is the concept of intra module.	Coupling is the concept of inter module.
2	Cohesion represents the relationship within module.	Coupling represents the relationships between modules.
3	Increasing in cohesion is good for software.	Increasing in coupling is avoided for software.
4	Cohesion represents the functional strength of modules.	Coupling represents the independence among modules.
5	High cohesion gives the best software.	Loose coupling gives the best software.
6	In cohesion, module focuses on the single thing.	In coupling, modules are connected to the other modules.